

UNIVERSIDAD DE GRANADA

Departamento de Ciencias de la Computación e Inteligencia Artificial

Práctica: Contenedores lineales y no lineales

Diccionario a partir de los tipos <list> y <set>
Guía de Teléfonos a partir del tipo map

(Práctica puntuable para la evaluación contínua))

Dpto. Ciencias de la Computación e Inteligencia Artificial
E.T.S. de Ingenierías Informática y de Telecomunicación
Universidad de Granada

Estructuras de Datos

Grado en Ingeniería Informática
Doble Grado en Ingeniería Informática y Matemáticas
Doble Grado en Ingeniería Informática y ADE

1.- Introducción

Los objetivos de este guión de prácticas son los siguientes:

1. Practicar con T.D.A desde la perspectiva de la STL.
2. Construir un TDA Diccionario con el tipo <list> y el tipo <set> y un TDA Guía de teléfonos con el tipo <map>

Los requisitos para poder realizar esta práctica son:

1. Haber estudiado el Tema de T.D.A. con la STL.

2.- Ejercicios

Se han de implementar (y documentar con Doxygen) tres tipos de datos abstractos:

- Diccionario: Se hará dos implementaciones.
  - En la primera el tipo que represente al diccionario será una lista de la STL (list). La clase diccionario a su vez será una clase plantilla de forma que podamos instaciarla por ejemplo para mantener el conjunto de palabras y sus definiciones, o podría instanciarse para ser un clase que traslade palabras del idioma español al idioma inglés. La instanciación que hagamos definirá su uso final.
  - En la segunda el tipo que representa al diccionario será un set de string. En este caso no nos interesará almacenar información asociada a la palabra clave. Además no será una clase plantilla.

- Guía de Teléfonos: Almacenaremos parejas de nombres y teléfonos. Para ellos usaremos la clase map de la STL, en el que el nombre es la clave y la información asociada es el teléfono.

2.1 Crear el TDA Diccionario usando el tipo list de la STL

Definición.- Los objetos que se almacenan en el diccionario son parejas de tipo T y de lista de tipo U. T es el tipo de dato asociado a una clave que no se repite (DNI p.ej.) y list<U> es una lista de datos (string p.ej) asociados a la clave de tipo T. El diccionario está ordenado de menor a mayor clave.

Para almacenar estas parejas usaremos el siguiente registro plantilla, que almacena la *clave* de tipo T y la información asociada será una lista.

```
template <class T,class
U>
struct data{
    T clave;
    list<U> info_asoci;
 };
```

La siguiente sobrecarga del operador menos, nos va a permitir saber cuando dos objetos de tipo data uno es menor que el otro. Este operador es necesario para luego mantener las entradas en el diccionario ordenadas por clave

```
template <class T, class U>
bool operator< (const data<T,U> &d1,const data
<T,U>&d2){
         if (d1.clave<d2.clave)
           return true;
         return false;
}
```

Definición.-Un diccionario es una lista de datos (parejas formadas por clave e información asociada), como hemos definidos anteriormente. El diccionario mantiene la información ordenada por clave y no puede haber claves repetidas.

Cuidado porque se manejan listas de listas. Se añaden dos funciones privadas que hacen más fácil la implementación de algunos operadores o funciones de la parte pública. Así una de ellas es el método privado *Copiar* que copia un diccionario en otro y *Borrar* que borra/elimina todos los elementos de un diccionario.

La implementación de la copia puede hacerse usando iteradores o directamente usando la función *assign* de la clase list.

A continuación se muestra la interfaz y representación de la clase Diccionario:

```
1.  template <class T,class U>
2.  class Diccionario{
3.    private:
4.      list<data<T,U> > datos;
5.      void Copiar(const Diccionario<T,U>& D); //Copia un diccionario
6.      void Borrar();  //Borra un diccionario
7.    public:
8.      Diccionario();  //Constructor por defecto
9.      Diccionario(const Diccionario &D); //Constructor de copias
10.     ~Diccionario(); //Destructor
11.     Diccionario<T,U> & operator=(const Diccionario<T,U> &D); //asignación
12.    /* Clases iteradoras */
13.    class iterator{
14.            private:...
15.            prublic:...
16.      }
```

```
17.          class const_iterator{
18.                private:
19.                      …
20.                public:
21.          }
```

```
     /* Busca la clave p en el diccionario. Si está devuelve un iterador a dónde
está   clave. Si no está, devuelve end() y deja el iterador de salida apuntando
al sitio  dónde  debería estar la clave */
22.         bool Esta_Clave(const T &p, typename  Diccionario::iterator
                                                                    &it_out);
```

```
          /* Inserta un nuevo registro en el diccionario. Lo hace a través de la
clave  e    inserta la lista con   toda la información asociada a esa clave. Si
el  diccionario no estuviera ordenado habría que usar la función sort()  */
23.        void Insertar(const T& clave,const list<U> &info);
```

```
         /*Añade una nueva informacion asocida a una clave que está en el
diccionario. La nueva información  se inserta al final de la lista de
información. Si no esta la clave la inserta y añade la
                informacion asociada. */
24.       void AddSignificado_Palabra(const U & s ,const T &p);
```

```
         /* Devuelve la información asociada a la  clave p*/
25.       list<U>  getInfo_Asoc(const T & p);
```

```
        /*Devuelve el tamaño del diccionario*/
26.     int size() const;
```

```
        /*Funciones begin() y end() asociadas al diccionario*/
….
}
```

Se pide:

1. Añadir la ***clase iterator y const_iterator*** dentro de Diccionario. Con esta clase iteradora podremos escribir, por ejemplo, códigos que nos permita recorrer el diccionario como el siguiente:

```
Diccionario<string,string> midiccionario;
Diccionario<string,string>::iterator iter;
….
for (iter = midiccionario.begin(); iter!
=midiccionario.end();++iter)
   cout«(*iter).clave« (*iter).info_asoci.size()«endl;
```

2. Implementar todos los métodos descritos en d**iccionario.h** en diccionario.cpp
3. Crear el programa usodiccionario que es el ejecutable asociado al fichero usodiccionario.cpp.
4. Documentar todos los métodos con doxygen

2.2 Crear el TDA Dictionary usando el tipo set de la STL

El TDA que implementaremos será el TDA Dictionary. Este TDA nos permitirá mantener en nuestro programa un conjunto de palabras. Como no estaremos interesados en almacenar las definiciones de nuestras palabras, sólo los términos, necesitaremos una estructura que nos permita almacenar strings, sin necesidad de almacenar información más compleja. Además, nos interesará que nuestro conjunto tenga dos propiedades específicas sobre sus elementos:

1 Los elementos deben estar ordenados por orden alfabético. Tiene mucho sentido que, en un diccionario, los elementos se ordenen de esta manera, y no en otro orden, independientemente del orden en que se hayan insertado

2 Los elementos no se deben repetir. Estamos manteniendo una lista de palabras de un idioma, no nos interesa tener palabras repetidas, si no saber simplemente qué palabras pertenecen al mismo.

La estructura de datos ***set*** es muy adecuada para lo que necesitamos, teniendo en cuenta las condiciones que hemos impuesto, así que será el contenedor que utilizaremos para nuestro TDA. Tendremos, por tanto, la siguiente representación para Dictionary:

```
class Dictionary {
private:
      set <string> words;
public:
…
}
```

Para esta clase, implementaremos la siguiente funcionalidades:

- Constructores por defecto y de copia

◆ Dictionary() [1/2]

Dictionary::Dictionary ( ) inline

Constructor por defecto.

Crea un **Dictionary** vacío

◆ Dictionary() [2/2]

Dictionary::Dictionary (const **Dictionary** & other) inline

Constructor de copia.

Crea un **Dictionary** con el mismo contenido que el que se pasa como argumento

Parameters

other **Dictionary** que se quiere copiar

- Métodos para insertar, consultar si existe un valor en el diccionario, y borrar un elemento

◆ exists()

bool Dictionary::exists (const string & val) const inline

Indica si una palabra esta en el diccionario o no.

Este método comprueba si una determinada palabra se encuentra o no en el dicccionario

Parameters

palabra la palabra que se quiere buscar.

Returns

Booleano indicando si la palabra existe o no en el diccionario

◆ insert()

bool Dictionary::insert (const string & val) inline

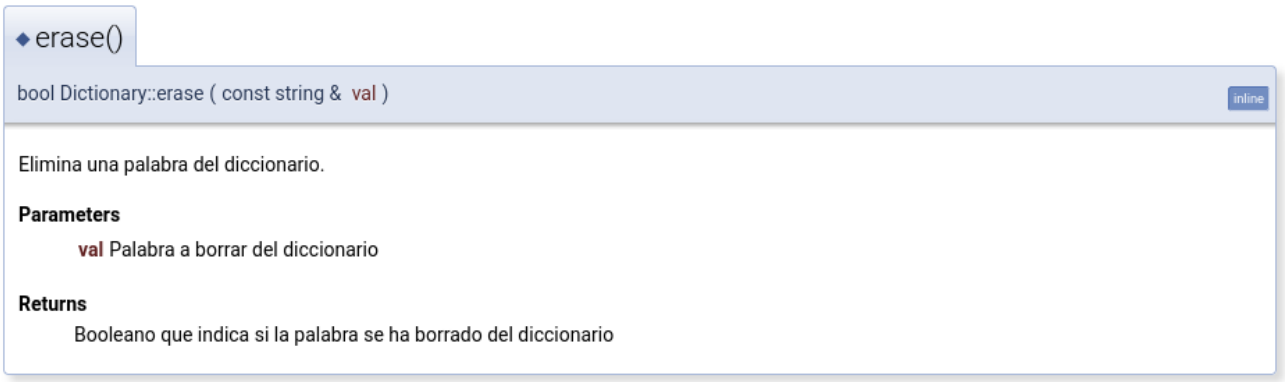
Inserta una palabra en el diccionario.

Parameters

val palaba a insertar en el diccionario

Returns

Booleano que indica si la inserción ha tenido éxito. Una palabra se inserta con éxito si no existía previamente en el diccionario



◆ erase()

bool Dictionary::erase (const string & val) inline

Elimina una palabra del diccionario.

Parameters

val Palabra a borrar del diccionario

Returns

Booleano que indica si la palabra se ha borrado del diccionario

- Métodos para consultar el tamaño, consultar si el diccionario está vacío, y limpiar el diccionario



◆ clear()

void Dictionary::clear ( ) inline

Limpia el Dictionary.

Elimina todas las palabras contenidas en el conjunto



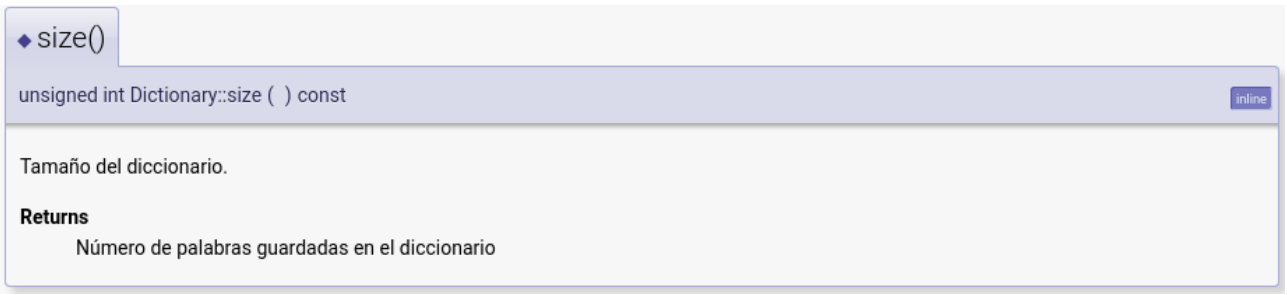
◆ empty()

bool Dictionary::empty ( ) const inline

Comprueba si el diccionario está vacio.

Returns

true si el diccionario está vacío, false en caso contrario



◆ size()

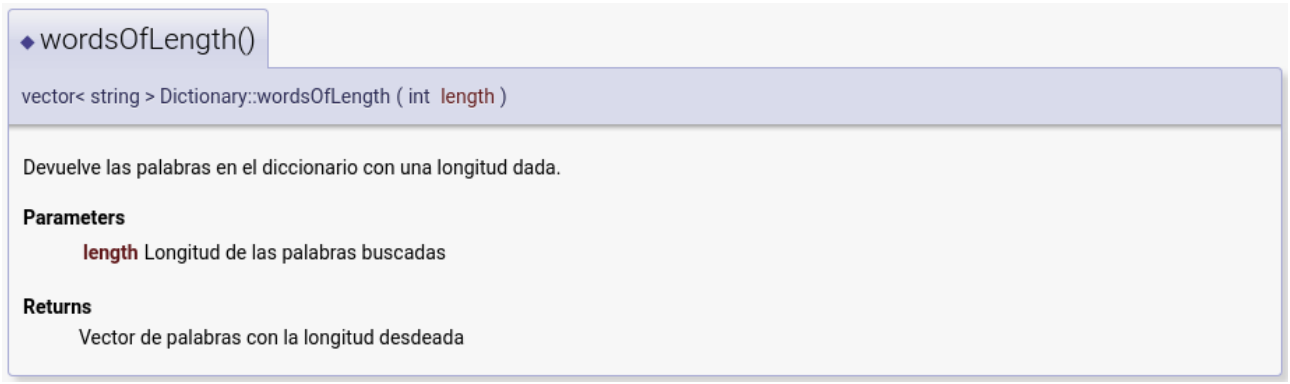
unsigned int Dictionary::size ( ) const inline

Tamaño del diccionario.

Returns

Número de palabras guardadas en el diccionario

- Obtener todas las palabras en el diccionario con una determinada longitud.



◆ wordsOfLength()

vector< string > Dictionary::wordsOfLength (int length)

Devuelve las palabras en el diccionario con una longitud dada.

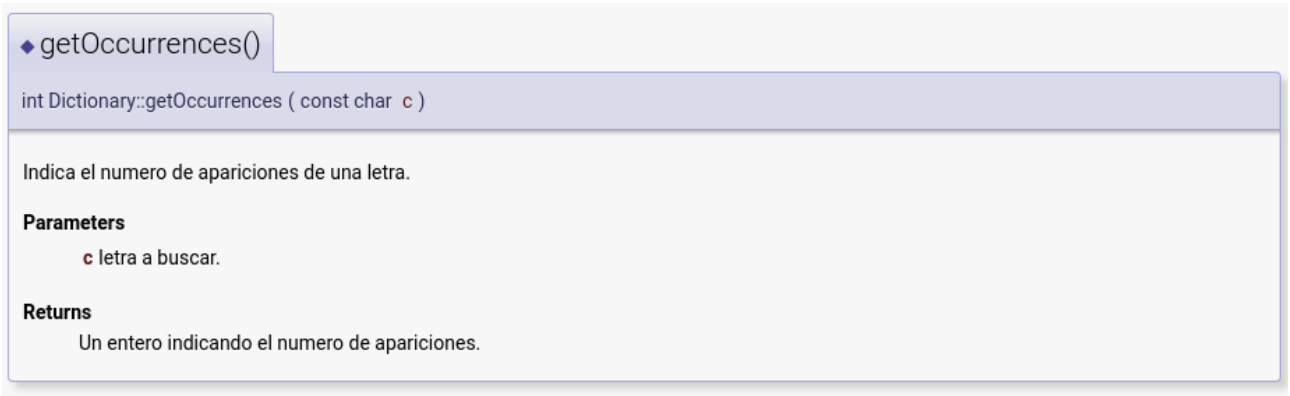
Parameters

length Longitud de las palabras buscadas

Returns

Vector de palabras con la longitud desdeada

- Número de veces que aparece un determinado carácter en el diccionario.



◆ getOccurrences()

int Dictionary::getOccurrences (const char c)

Indica el numero de apariciones de una letra.

Parameters

c letra a buscar.

Returns

Un entero indicando el numero de apariciones.

Sobre esta clase se pide

- Implementar los métodos de las clase Dictionary, la interfaz la tenéis en dictionary.h. Los métodos se deben implementar en dictionary.cpp

Además, para probar el funcionamiento de este TDA, implementaremos tres programas de prueba distintos.

- El primero de ellos, ***palabras_longitud.cpp,*** recibe dos argumentos
  - Un fichero con las palabras de un diccionario
  - Un entero con la longitud de las palabras que buscamos

  El funcionamiento de este programa será: 1) Construir un Dictionary con el fichero de palabras que le indica el usuario desde la linea de ordenes; 2) Extrae de dicho diccionario las palabras de la longitud que buscamos; 3) Imprime por pantalla las palabras. Una posible invocación al programa es la que se muestra a continuación:

```
prompt> ./palabras_longitud data/diccionario.txt 5
```

- El segundo de ellos, ***apariciones.cpp,*** recibe dos argumentos
  - Un fichero con las palabras de un diccionario

- Un carácter.

Este programa devuelve el número de veces que ese carácter aparece en el diccionario. Un posible invocación del programa desde la líneas de órdenes es la siguiente:

```
prompt> ./apariciones data/diccionario.txt a
```

En este ejemplo queremos obtener el número de apariciones en el diccionario del carácter a.

- El tercero, ***union.cpp,*** recibe dos argumentos
  - Un fichero con las palabras de un diccionario.
  - Otro fichero de diccionario.

  Este programa genera un nuevo diccionario resultado de la unión de los diccionarios dados en la entrada. Una posible llamada sería:

```
prompt> ./union data/diccionario.txt data/diccionario_pub.txt
```

2.3 Crear el TDA Guia de Teléfonos usando el tipo map de la STL

Se trata de crear el TDA **Guia_Tlf** basándose en el tipo **map** de la STL.

Definición. Una guía de teléfonos contiene una colección de parejas <nombre, num_teléfono> . Siendo nombre la clave y num_teléfono la información asociada. La guía de teléfonos esta ordenada por nombre y no puede existir nombres repetidos.

Usaremos para representar el nombre y el número de teléfono el tipo string. Para representar la guía usaremos el tipo map de la STL. En los ficheros include/guiatlf.h src/guatlf.cpp se da la clase Guia_Tlf.

Se pide:

1. Implementar los métodos de la clase Guia_Tlf. Para ello tenéis la interfaz en guiatlf.h y la implementación la debéis hacer en guiatlf.cpp.
2. Añadir una clase iteradora, además de los métodos begin() y end().
3. Para probar el funcionamiento de estos los tres métodos descritos en el punto 3 implementaremos tres programas de prueba.
   - El primero de ellos, ***inter.cpp***, recibe dos argumentos
     - Un fichero con una guía.
     - Otro nombre de fichero con otra guía de teléfonos.

   El programa debe construir una nueva guía con la intersección de las dos pasadas por argumento e imprimirla por pantalla.

- El segundo programa de prueba se llama ***modificar.cpp*** que modifica el teléfono asociado a un nombre. Recibe tres argumentos.
  - Un fichero con una guía.
  - El nombre al que le vamos a modificar el teléfono.
  - El nuevo teléfono.

  Debe buscar el nombre en la guía y modificar el teléfono por el nuevo introducido como argumento. Finalmente debe imprimir la nueva guía por pantalla.

- El tercero de los programas de prueba se llama ***filtro.cpp*** que recibe dos argumentos:
  - El fichero con una guía.
  - Un string que es el comienzo de un nombre.

  Debe mostrar por pantalla todos los nombres que comiencen por ese string con sus respectivos teléfonos.

Como resumen, la guia de teléfonos contiene como tipo base parejas <nombre, num_teléfono> ambos de tipo string

Para facilitar la lectura seria recomendable implemetar el operador» para leer una pareja de strings:

```
istream & operator>>(istream
&is,pair<string,string> &d){
       getline(is,d.first,'\t');
       getline(is,d.second);
       return is;
}
```

La clase y sus procedimientos básicos se definen como:

```
class Guia_Tlf{
       private:
         map<string,string> datos; //si se admite que haya nombres
                                   //repetidos se usará un multimap
        public:
           Guia_Tlf(){}
           //Guia_Tlf(const Guia_Tlf & gt);
```

```
            //~Guia_Tlf(){}
            //Guia_Tlf & operator=(const Guia_Tlf & gt);
/**
@brief Acceso a un elemento
@param nombre: nombre del elemento  elemento acceder
@return devuelve el valor asociado a un nombre, es decir el teléfono
  */
string & operator[](const string &nombre);
/**
@brief Insert un nuevo telefono
@param nombre: nombre clave del nuevo telefono
@param tlf: numero de telefono
@return : un pair donde first apunta al nuevo elemento insertado y bool es true si se ha insertado el nuevo tlf o false en caso contrario
 */
pair<map<string,string>::iterator,bool>     insert(string  nombre, string tlf);
/**
@brief Insert un nuevo telefono
 @param p: pair con el nombre y el telefono asociado
 @return : un pair donde first apunta al nuevo elemento insertado y bool es true si se ha insertado el nuevo tlf o false en caso contrario
 */
pair<map<string,string>::iterator,bool>
insert(pair<string,string> p);
/**
@brief Borrar un telefono
@param nombre: nombre que se quiere borrar
@note: en caso de que fuese un multimap borraria todos con ese nombre
 */
void borrar(const string &nombre);
```

```
/**
 @brief Union de guias de telefonos
@param g: guia que se une
@return: una nueva guia resultado de unir el objeto al que apunta this y g
 */
Guia_Tlf operator+(const Guia_Tlf & g);
/**
 @brief Diferencia de guias de telefonos
@param g: guia que se une
@return: una nueva guia resultado de la diferencia del objeto al que apunta this y g
 */
Guia_Tlf operator-(const Guia_Tlf & g);
/**
@brief Escritura de la guia de telefonos
@param os: flujo de salida. Es MODIFICADO
@param g: guia de telefonos que se escribe
@return el flujo de salida
 */
friend ostream & operator<<(ostream & os, Guia_Tlf & g);
/**
@brief Lectura de  la guia de telefonos
@param is: flujo de entrada. ES MODIFICADO
@param g: guia de telefonos. ES MODIFICADO
@return el flujo de entrada
*/
friend istream & operator>>(istream & is, Guia_Tlf & g);
/* Añadir  la clase iterator (mirad guiatlf.h para ver los métodos a implementar)*/
};
```

3.- Resumen de la práctica a Realizar

3.1 Construcción del TDA Diccionario usando el tipo <list>

En el segundo ejercicio de la práctica, se propone la implementación del **tipo de dato Diccionario**, que implementa el diccionario de letras que hemos descrito en los apartados anteriores. Para ello, será necesario:

1 Dar una especificación del T.D.A Diccionario

2 Implementar el TDA Diccionario **utilizando como contenedor subyacente un tipo <list>** teniendo en cuenta los principios de **ocultamiento de información**, se recomienda separar la declaración y la implementación de dicha clase (se sugieren los nombres de archivo diccionario.h y diccionario.cpp, situados en la carpeta que les corresponda)

3 Se deberán implementar iteradores para poder iterar sobre los diferentes elementos del TDA Diccioario

4 Hacer el programa usodiccionario que prueba el funcionamiento del TDA Diccionario. Para ello el estudiante dispone de usodiccionario.cpp.

3.2: Construcción del TDA Dictionary usando el tipo <set>

En el segundo ejercicio de la práctica, se propone la implementación del **tipo de dato Dictionary**, que implementa el diccionario de letras que hemos descrito en los apartados anteriores. Para ello, será necesario:

1. Dar una especificación del T.D.A Dictionary.
2. Implementar el TDA Dictionary **utilizando como contenedor subyacente un set.** Se podrá utilizar el set que viene implementado en la STL (set). Teniendo en cuenta los principios de **ocultamiento de información**, se recomienda separar la declaración y la implementación de dicha clase (se sugieren los nombres de archivo dictionary.h y dictionary.cpp, situados en la carpeta que les corresponda)
3. Se deberán implementar iteradores para poder iterar sobre los diferentes elementos del TDA Dictionary.
4. Hacer los programas de prueba **palabras_longitud.cpp, apariciones.cpp y union.cpp**

3.3 Construcción del TDA Guia_Tlf usando el tipo <map>

En el tercer ejercicio de la práctica, se da la implementación del **tipo de dato Guia_Tlf**, que implementa la guia de teléfonos que hemos descrito en los apartados anteriores. Será necesario:

1. Repasar los comentarios doxygen de todos los métodos.
2. Implementar todos los métodos de la clase Guia_Tlf **.**
3. Hacer los correspondientes tres programas de prueba para probar algunos de los métodos de la guía.
4. Se deberán **implementar iteradores** para poder iterar sobre los diferentes elementos del TDA Guia_Tlf.

Hacer la página principal de la documentación común para todos los TDAs,

4.- Uso de LLM's

Para la elaboración de la práctica, podrán usarse los LLM's que cada pareja considere en los apartados que estimen oportuno. La condición es documentar su uso en aquellas partes en que se hayan utilizado explicando cómo y por qué se han usado. Un LLM es una herramienta más a disposición de un ingeniero informático, pero solo es eso: una herramienta. No hay ni que minusvalorla ni tampoco sobrevalorarla. En esta práctica puede ser un apoyo, pero no el eje de la solución.

5.- Documentación y entrega

Toda la documentación de la práctica se incluirá en el propio Doxygen generado, para ello se utilizarán tanto las directivas Doxygen de los archivos .h y .cpp como los archivos .dox incluídos en la carpeta doc.

1. La entrega no es obligatoria pero si se desea entregar hacer un .zip con lo siguiente:
   - Se dispone de la siguiente estructura de ficheros:
     - /
       - estudiante/ (Aquí irá todo lo que desarrolle el alumno)
         - doc/ (Imágenes y documentos extra para Doxygen)
         - include/ (Archivos cabecera)
         - src/ (Archivos fuente)
       - data/ (Datos de ejemplo)
       - CMakeList.txt (Instrucciones CMake)
       - Doxyfile.in (Archivo de configuración de Doxygen)

2. Elaboración y puntuación
   - La práctica se realizará POR PAREJAS. Los nombres completos se incluirán en la descripción de la entrega en Prado. Los DOS integrantes de la pareja deben subir el archivo a Prado, para así responsabilizarse ambos de su contenido y entrega.
   - Se trata de una práctica voluntaria puntuable.para la **evaluación contínua** de la asignatura. **su valor máximo es de 0.4 puntos.**
   - La fecha límite de entrega aparecerá en la subida a Prado.

6.- Referencias

- Rosa Rodriguez-Sánchez, J. Fdez-Valdivia, J.A. García, Estructuras de Datos en C++. Un enfoque práctico, 2020
- Garrido, A. Fdez-Valdivia, J. "*Abstracción y estructuras de datos en C++*". Delta publicaciones, 2006.